

Computer First:

```
#include <stdio.h>
#include <conio.h>
#include <ctype.h>

int numProductions;
char lhs[20];
char rhs[20][20];
char result[20];
int resultIdx;

void findFirst(char c);
void addToResult(char c);

int main() {
    int i, j;
    char choice;
    printf("Enter number of productions: ");
    scanf("%d", &numProductions);
    printf("Enter productions (e.g., E=TR or E=# for epsilon):\n");
    for (i = 0; i < numProductions; i++) {
        scanf(" %c=%s", &lhs[i], rhs[i]);
    }
    do {
        printf("\nEnter a Non-Terminal to find its FIRST set: ");
        scanf(" %c", &choice);
        resultIdx = 0;
        findFirst(choice);

        printf("FIRST(%c) = { ", choice);
        for (i = 0; i < resultIdx; i++) {
            printf("%c ", result[i]);
        }
        printf("}\n");
        printf("Do you want to continue? (y/n): ");
        scanf(" %c", &choice);
    } while (choice == 'y' || choice == 'Y');
    getch();
    return 0;
}

void findFirst(char c) {
    int i;

    if (!isupper(c)) {
        addToResult(c);
        return;
    }
}
```

```

for (i = 0; i < numProductions; i++) {
    if (lhs[i] == c) {
        if (rhs[i][0] == '#') {
            addToResult('#');
        }
        else if (!isupper(rhs[i][0])) {
            addToResult(rhs[i][0]);
        }
        else {
            findFirst(rhs[i][0]);
        }
    }
}
}

void addToResult(char c) {
    int i;
    for (i = 0; i < resultIdx; i++) {
        if (result[i] == c) {
            return;
        }
    }
    result[resultIdx++] = c;
}

```

Output:

```

Enter number of productions: 5
Enter productions <e.g., E=TR or E=# for epsilon>:
A=bc
A=BC
B=d
B=#
C=i

Enter a Non-Terminal to find its FIRST set: A
FIRST<A> = { b d # }
Do you want to continue? <y/n>:

```

Compute Follow:

```
#include <stdio.h>
#include <conio.h>
#include <ctype.h>
#include <string.h>

int numProductions;
char lhs[20];
char rhs[20][20];
char result[20];
int resultIdx;

void findFollow(char c);
void findFirstOfChar(char c);
void addToResult(char c);

int main() {
    int i;
    char choice;
    printf("Enter number of productions: ");
    scanf("%d", &numProductions);

    printf("Enter productions (e.g., E=TR or E=# for epsilon):\n");
    for (i = 0; i < numProductions; i++) {
        scanf(" %c=%s", &lhs[i], rhs[i]);
    }

    do {
        printf("\nEnter a Non-Terminal to find its FOLLOW set: ");
        scanf(" %c", &choice);

        resultIdx = 0;

        if (choice == lhs[0]) {
            addToResult('$');
        }
        findFollow(choice);
        printf("FOLLOW(%c) = { ", choice);
        for (i = 0; i < resultIdx; i++) {
            printf("%c ", result[i]);
        }
        printf("}\n");

        printf("Do you want to continue? (y/n): ");
        scanf(" %c", &choice);
    } while (choice == 'y' || choice == 'Y');
```

```

    getch();
    return 0;
}

```

```

void findFollow(char c) {
    int i, j;

    for (i = 0; i < numProductions; i++) {
        int length = strlen(rhs[i]);

        for (j = 0; j < length; j++) {
            if (rhs[i][j] == c) {
                if (j + 1 < length) {
                    char nextSymbol = rhs[i][j + 1];
                    if (!isupper(nextSymbol)) {
                        addToResult(nextSymbol);
                    }

                    else {
                        findFirstOfChar(nextSymbol);
                    }
                }
                if (j + 1 == length && lhs[i] != c) {
                    findFollow(lhs[i]);
                }
            }
        }
    }
}

```

```

void findFirstOfChar(char c) {
    int i;
    if (!isupper(c)) {
        addToResult(c);
        return;
    }
    for (i = 0; i < numProductions; i++) {
        if (lhs[i] == c) {
            if (rhs[i][0] != '#') {
                if (!isupper(rhs[i][0])) {
                    addToResult(rhs[i][0]);
                } else {
                    findFirstOfChar(rhs[i][0]);
                }
            }
        }
    }
}

```

```

}

void addToResult(char c) {
    int i;
    if (c == '#') return;
    for (i = 0; i < resultIdx; i++) {
        if (result[i] == c) {
            return;
        }
    }
    result[resultIdx++] = c;
}

```

Output:

```

Enter number of productions: 5
Enter productions (e.g., E=TR or E=# for epsilon):
A=bc
A=BC
B=b
B=#
C=f

Enter a Non-Terminal to find its FOLLOW set: A
FOLLOW(A) = { $ }
Do you want to continue? (y/n): y

Enter a Non-Terminal to find its FOLLOW set: B
FOLLOW(B) = { f }
Do you want to continue? (y/n):

```